RISC-V Privilege Modes & Interrupts

Lab 5 Tutorial

CSL251 COA Lab

February 9, 2026

Outline

1. Introduction
2. Control and Status Registers (CSRs)
3. CSR Instructions
4. System Instructions
5. Key Registers
6. General Instructions

Lab Objective

- Understand Privilege Mode Switching (Machine Mode $\leftrightarrow$ User Mode)
- Handle Exceptions (System Calls)
- Handle Interrupts (Timer Interrupts)
- Use the **Spike** Simulator for bare-metal debugging

RISC-V Privilege Modes

RISC-V defines three main privilege modes to isolate software components:

- **Machine Mode (M-Mode, 3):** Highest privilege. Full access to memory, I/O, and control registers. The bootloader/firmware runs here.
- **Supervisor Mode (S-Mode, 1):** Operating System (kernel) level. Used for virtual memory management and page tables.
- **User Mode (U-Mode, 0):** Lowest privilege. Applications run here. Cannot access CSRs or sensitive instructions directly (must use `ecall`).

Control and Status Registers (CSRs) in RISC-V

Control and Status Registers (CSRs) are special system registers in RISC-V used to control and monitor the processor state. Unlike general-purpose registers, CSRs manage low-level system functions such as:

- Handling exceptions and interrupts
- Switching privilege modes
- Checking processor configuration
- Enabling or disabling interrupts

CSRs can be read, written, and in many cases, individual bits can be set or cleared.

Why CSRs are Needed

CSRs are used when normal registers are insufficient. Typical use cases include:

- Detecting why a trap occurred
- Saving the address of a faulting instruction
- Enabling timer, software, or external interrupts
- Jumping to trap handlers

CSRs cannot be accessed like normal registers. Instead, RISC-V provides special CSR instructions.

CSR Instructions (Control Status Registers)

Instructions to read/write special system registers.

csrr (CSR Read)

`csrr rd, csr`

Reads a CSR value into a general-purpose register.

Example: `csrr t0, mstatus`

csrw (CSR Write)

`csrw csr, rs`

Writes a value from a general-purpose register to a CSR.

Example: `csrw mepc, t1`

csrs (CSR Set Bit)

`csrs csr, rs`

Sets specific bits in a CSR (bitwise OR).

Example: `csrs mie, t0`

System Instructions

ecall (Environment Call)

- Used by User-Mode code to request service from Machine-Mode.
- Triggers a **Synchronous Exception**.
- Jumps to the trap handler defined in `mtvec`.

mret (Machine Return)

- Returns from a Machine-Mode trap.
- Sets pc $\leftarrow$ `mepc`
- Sets Privilege Mode $\leftarrow$ `mstatus.MPP`

Important CSRs (1/2)

mstatus (Machine Status)

Tracks global system state.

- **MIE (Bit 3):** Machine Interrupt Enable.
- **MPP (Bits 11-12):** Machine Previous Privilege. Controls which mode `mret` returns to (11=Machine, 00=User).

mepc (Machine Exception Program Counter)

- Stores the address where the trap occurred.
- `mret` uses this address to return.
- **Note:** For `ecall`, we often need to modify this to avoid re-executing the same instruction (or not, depending on logic).

Important CSRs (2/2)

mtvec (Machine Trap-Vector Base-Address)

- Stores the address of your Trap Handler function.

mcause (Machine Cause)

Indicates *why* the trap occurred.

- 8: User `ecall`
- `Interrupt bit + 7`: Machine Timer Interrupt

mie (Machine Interrupt Enable)

- Individual enable bits for specific interrupts.
- **MTIE (Bit 7):** Timer Interrupt Enable.

General Assembly Instructions

- **li (Load Immediate):**
  `li t0, 500000` (Load constant)
- **la (Load Address):**
  `la t0, mtrap` (Load label address)
- **ld / sd (Load/Store Doubleword):**
  `sd t0, 0(t1)` (Memory access)
- **slli (Shift Left Logical Immediate):**
  `slli t1, t1, 11` (Bit manipulation)